

USS

USS-ICIFG003

Clase 07

Clase 07 — CI/CD

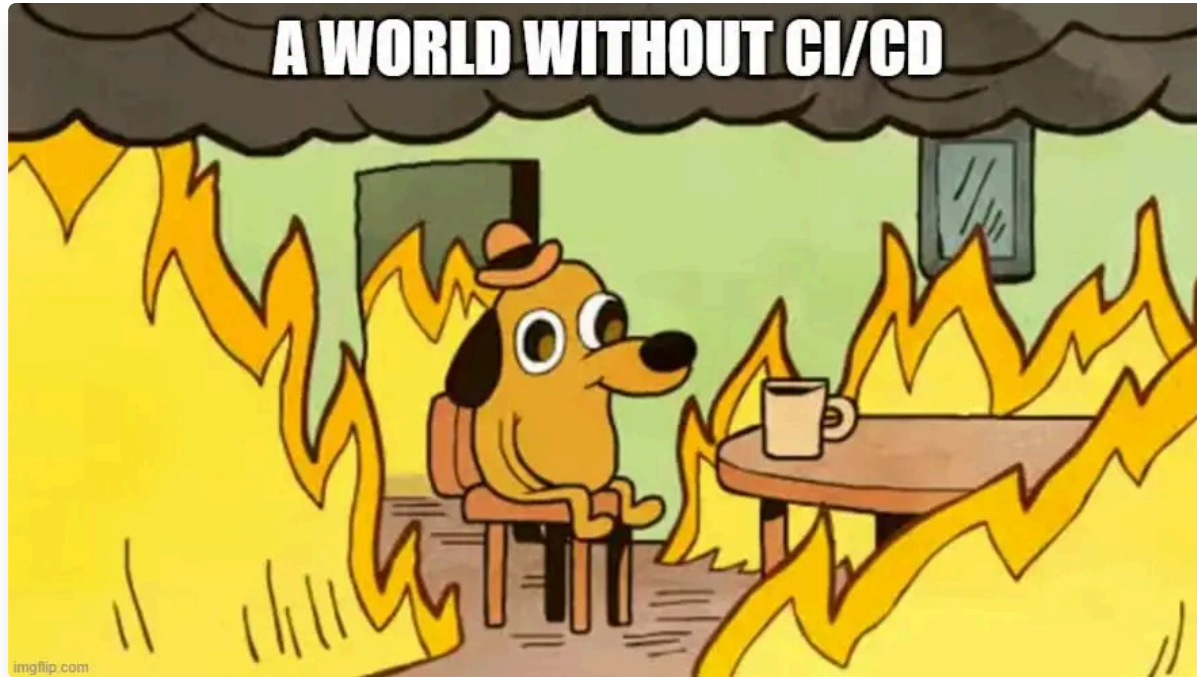
Integración y entrega continua con GitHub Actions.

2026-04-03

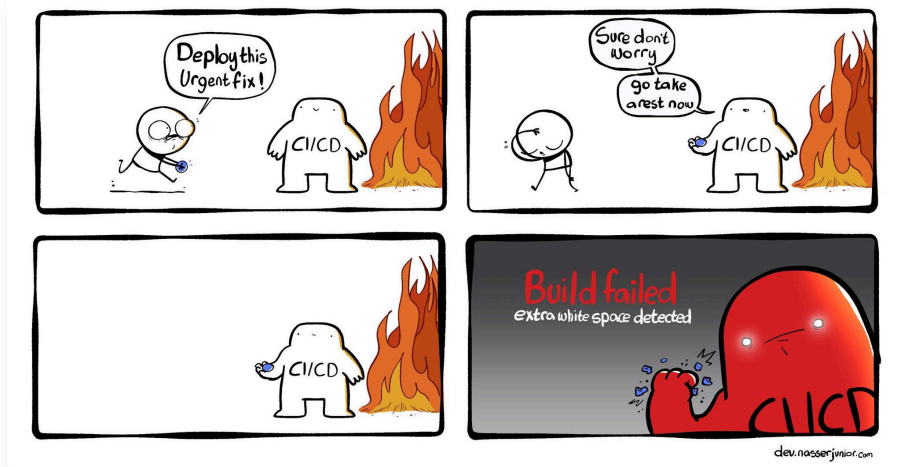
Tabla de Contenidos

1. [CI/CD y GitHub Actions](#)
2. [Introducción a GitHub Actions](#)
3. [Probando en Local: Act](#) 
4. [Conclusiones](#)

Caos

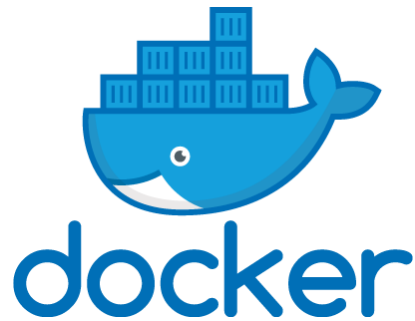


Los extremos nunca son buenos



Clase anterior

- Contenedores
 - Docker
 - DockerHub
 - Docker Compose
 - Registros



Ejemplo Dockerfile

```
1 FROM python:3.9
2
3 WORKDIR /app
4
5 # Es necesario agregar /app?
6 COPY . /app
7
8 RUN pip install --no-cache-dir -r requirements.txt
9
10 EXPOSE 80
11
12 # Que es esto?
13 ENV NAME World
14
15 CMD ["python", "app.py"]
```

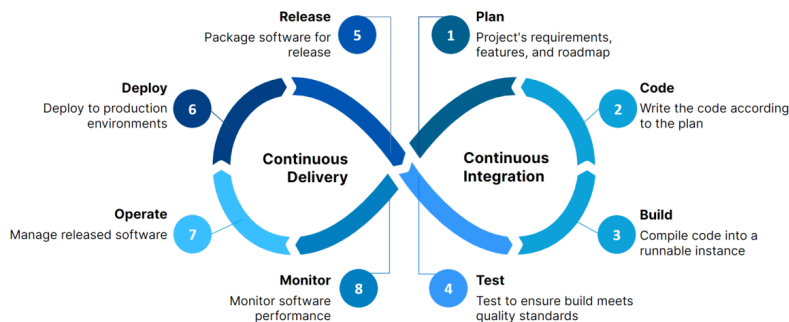
Integración y entrega continua.

CI/CD y GitHub Actions

CI/CD y GitHub Actions

La Importancia de CI/CD (Continuous Integration/Continuous Delivery)

- **Integración Continua (CI):** Fusión regular de código + construcción y pruebas automatizadas.
- **Entrega Continua (CD):** Preparación automatizada para lanzamiento a producción (extensión de CI).
- **Tendencia:** Integraciones frecuentes para minimizar problemas y fomentar la colaboración.



Beneficios de CI/CD

- **Detección temprana de errores:** Pruebas automatizadas con cada cambio.
- **Ciclos de retroalimentación rápidos:** Información casi instantánea para los desarrolladores.
- **Mayor calidad del código:** Cumplimiento de estándares y prevención de regresiones.
- **Reducción del esfuerzo manual:** Automatización de tareas repetitivas.

Introducción a GitHub Actions

Plataforma de CI/CD Integrada en [GitHub](#)

- **Automatización de flujos de trabajo:** Desde la idea hasta la producción.
- **Beneficios clave:**
 - Automatización de tareas repetitivas.
 - Integración perfecta con repositorios.
 - Flexibilidad para personalizar flujos de trabajo.
- **Automatización de cualquier webhook (eventos):** Amplio abanico de posibilidades.

Anatomía de un Workflow

Un **workflow** es un archivo YAML (*.yaml*) que reside en la carpeta *.github/workflows/* y automatiza procesos.

Su estructura sigue una jerarquía clara:

Término	Nivel	Significado
Workflow	□	El archivo completo.
Activador	↘	Evento que dispara el flujo (<i>on</i>).
Jobs	□	Grupos de pasos (paralelos).
Steps	□	Tareas individuales (secuenciales).
Runner	□	Servidor virtual (ej: Ubuntu).

```
Workflow (.yaml)
├─ Activador (on: push, etc.)
├─ Jobs (paralelos)
│   └─ Job 1 (Runner)
│       └─ Step 1 (secuencial)
│           └─ Action / Command
│       └─ Step 2
│           └─ Action / Command
└─ Job 2 (Runner)
    └─ Step 1
        └─ Action / Command
```

0. El nombre del Workflow

Es opcional pero recomendado para identificarlo fácilmente en la interfaz de GitHub.

- Recuerden utilizar nombres descriptivos y claros. Es importante la semantica para la colaboración y mantenimiento a largo plazo.

```
1 name: Python Quality Control
```

1. El Activador (*on*)

Define **cuándo** se debe ejecutar el flujo de trabajo.

- **Push:** Cada vez que subes código.
- **Pull Request:** Cuando alguien propone cambios.
- **Schedule:** Como una "tarea programada" (cron).
- **Workflow Dispatch:** Botón manual en GitHub.
- ... [docs](#) para más eventos.

```
1  name: Mi Workflow
2  on:
3    push:
4      branches: [ main ]
5    pull_request:
6      branches: [ main ]
7    workflow_dispatch:
```

2. Los Trabajos (*jobs*)

Los *jobs* corren en paralelo por defecto (a menos que se indique lo contrario).

- **id:** Nombre técnico del trabajo (*build*).
- **runs-on:** Sistema operativo del servidor virtual.
- **steps:** La lista de acciones a realizar en orden.

```
1  jobs:
2    build:
3      name: Compilación y Pruebas
4      runs-on: ubuntu-latest
5      steps:
6        - name: Saludo
7          run: echo "Iniciando..."
8    lint:
9      name: Análisis de Código
10     runs-on: ubuntu-latest
11     steps:
12       - name: Saludo
13         run: echo "Analizando código..."
```

3. Pasos y Acciones

Es una combinación de comandos directos y *Actions* pre-hechas (plugins reutilizables).

- **uses:** Utiliza una "Action" pre-hecha (como un plugin).
 - *actions/checkout*: Descarga tu código al servidor.
- **run:** Ejecuta un comando de consola directamente.
- **name:** (Opcional) Título amigable para el log de GitHub.

```
1  steps:
2    - name: Descargar código
3      uses: actions/checkout@v4
4
5    - name: Instalar Python
6      uses: actions/setup-python@v5
7      with:
8        python-version: '3.12'
9
10   - name: Ejecutar Tests
11     run: pytest
```

Similar a como trabajamos en Dockerfile, pero con la flexibilidad de usar acciones pre-hechas o comandos personalizados.

Uniendo todo (*[.github/workflows/ci.yml](#)*)

```
1  name: Python Quality Control
2  on: [push, pull_request]
3
4  jobs:
5    test:
6      runs-on: ubuntu-latest
7      steps:
8        - uses: actions/checkout@v4
9
10       - name: Set up Python
11         uses: actions/setup-python@v5
12         with:
13           python-version: '3.12'
14
15       - name: Install dependencies
16         run: |
17           python -m pip install --upgrade pip
18           pip install -r requirements.txt
19
20       - name: Run Pytest
21         run: pytest
```


Pasos Finales

1. **Guardar y enviar:** Commit y push del archivo de workflow.
2. **Detección automática:** GitHub detecta el nuevo archivo.
3. **Activación:** En respuesta a los eventos definidos en *on*.
4. **Monitorización:** Pestaña *Actions* en la página del repositorio.
 - Estado de los workflows.
 - Detalles de cada ejecución (logs, salidas).

Consejos para workflows de pruebas

- **Workflows concisos:** Centrados en tareas específicas.
- **Nombres descriptivos:** Workflows, jobs y steps.
- **Almacenamiento en caché:** Acelerar ejecuciones (*actions/cache*).
 - Almacena dependencias entre ejecuciones.
 - Reduce el tiempo de descarga e instalación.

Matrix Builds: Pruebas Multientorno

Permite ejecutar múltiples trabajos con diferentes combinaciones de variables.

- **Útil para:** Probar en varias versiones de Python, SO o dependencias simultáneamente.
- **Eficiencia:** GitHub crea un contenedor distinto por cada combinación.

```
1  jobs:
2    test:
3      runs-on: ${ matrix.os }
4      strategy:
5        matrix:
6          os: [ubuntu-latest, windows-latest]
7          python-version: ['3.10', '3.12']
8      steps:
9        - uses: actions/checkout@v4
10       - uses: actions/setup-python@v5
11         with:
12           python-version: ${ matrix.python-version }
13       - run: pip install -r requirements.txt
14       - run: pytest
```

Badges: Secretos y Variables de Entorno

Son herramientas clave para mejorar la visibilidad y seguridad de tus workflows.

- *Badges*: Indicadores visuales del estado del workflow en el README.
- *Secretos*: Almacenan información sensible (tokens, claves) de forma segura.
- *Variables de entorno*: Configuraciones que pueden variar entre entornos (dev, prod).

Por ejemplo, son necesarios para desplegar a producción sin exponer credenciales en el código.

Probando en Local: Act

[act](#) es una herramienta que permite simular GitHub Actions en tu propia máquina usando Docker.

- **Evita el ciclo:** *Commit -> Push -> Fallo en CI -> Fix -> Push*.
- **Ahorra minutos:** No dependes de la cola de GitHub.
- **Privacidad:** Ideal para probar secretos o configuraciones sensibles.

Conclusiones

- **CI/CD** es una práctica esencial para la calidad y velocidad del software moderno.
- **GitHub Actions** ofrece una integración nativa y potente sin salir del repositorio.
- **Automatización = Confianza:** Menos errores manuales, despliegues más seguros.
- No es infinito: Recuerda los [límites de uso](#) (gratis para repos públicos).

¿Preguntas?

¿Tienen alguna pregunta?